

Human-Extractable ZK Proofs of Knowledge: A Solution to Dark DAOs

Zeyuan Yin^{1,2}, Leiyan Tian^{1,2}, Bingsheng Zhang^{1,2}, and Kui Ren^{1,2}

¹ The State Key Laboratory of Blockchain and Data Security, Zhejiang University
{zeyuanyin, lytian.cs, bingsheng, kuiren}@zju.edu.cn

² Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security

Abstract. A Decentralized Autonomous Organization (DAO) is a pioneering evolution to realize a decentralized democratic governance over a blockchain. In a DAO, stakeholders usually make collective decisions through secure on-chain voting. Recently, Dark DAO (Austgen et al., arXiv:2311.03530) was proposed as a decentralized cartel that enables automated vote-buying. It attacks the *inalienable authentication* of a remote e-voting system by leveraging key encumbrance via MPC or TEEs, enabling a voter to pass the authentication without knowing the actual key. To defend against this new type of attack, the notions of *individual knowledge* (Dziembowski et al., CRYPTO '23) and *complete knowledge* (Kelkar et al., CCS '24) were proposed, ensuring that the prover has unencumbered knowledge of a secret. However, their solutions rely on TEEs or ASICs, which are difficult to deploy on blockchain.

Inspired by the human-extractable CAPTCHA puzzles proposed by Kumarasubramanian et al. (PKC '13), we propose a new primitive called *human-extractable zero-knowledge proofs of knowledge* (HE-ZKPoK) as an alternative solution to Dark DAOs. Our HE-ZKPoK protocol forces the prover to solve human-extractable CAPTCHA puzzles along with completing a standard zero-knowledge proof of knowledge, avoiding the need for specialized hardware. As a result, any human entity can extract the witness merely by looking at the prover's CAPTCHA queries and the associated puzzles. Under the assumption that humans cannot encumber a secret, we conclude that if a voter sells his vote, his secret key will be fully exposed, thus deterring voters from engaging in vote-buying.

Keywords: Human-extractable CAPTCHA, Dark DAOs, Inalienable authentication, Human-extractable zero-knowledge proofs of knowledge.

1 Introduction

With the development of blockchain, Decentralized Autonomous Organizations (DAOs) are becoming increasingly popular. A DAO is an organization that operates based on self-executing rules deployed on a public blockchain [7, 19]. In contrast to a corporation whose decisions are made by a CEO and board of directors, a DAO's governance is decentralized to its stakeholders. Typically, a

DAO’s decision-making process involves voting on proposals via smart contracts, with each participant’s vote weighted by their stake.

A concern of DAO decision-making is that remote electronic voting is susceptible to coercion and vote-buying [20]. Unlike real-world voting, where a voting booth protects voters from coercion, in remote electronic voting, the voting procedure can be viewed as a probabilistic algorithm. Even if the ballot is encrypted, the input and randomness used in the voting procedure serve as a receipt for casting a particular ballot, and anyone can rerun the probabilistic algorithm to verify it. This creates the opportunity for coercion and vote-buying. To address this issue, several coercion-resistant voting schemes have been proposed in recent years [20,9,1,3,25,14]. They either allow voters to re-cast their ballots after being coerced [1,25,14] (*re-voting*) or enable voters to generate fake ballots that are indistinguishable to a coercer [20,9,3] (*fake credentials*). However, all coercion-resistant voting schemes assume that the authentication procedure is inalienable [1], i.e., the coercer can neither impersonate the voter nor prevent the voter from being authenticated. Unfortunately, a recent work by Austgen et al. [4] illustrates the risk of Dark DAO attacks, which can break the inalienable authentication assumption.

Dark DAOs. The concept of Dark DAOs was first proposed in the blog post [10] and was later demonstrated its practical realization in [4]. A Dark DAO aims to buy on-chain votes opaquely. Here, “opaquely” means that a Dark DAO member is indistinguishable from other valid participants. A Dark DAO also guarantees fair exchange, i.e., the bribee will be paid if and only if they vote as instructed. The main tool of a Dark DAO is key encumbrance [21], which can create a key that is designated for particular purposes. Key encumbrance can be realized by secure multi-party computation (MPC) or trusted execution environments (TEEs). For now, TEEs, such as Intel SGX [27,26], are the more practical choices. For example, in the case of voting, a Dark DAO can run a TEE to create an encumbered key sk , such that the briber can use sk only to vote while the bribee can use sk to manage their cryptocurrency. In this scenario, the traditional notion of proof of knowledge (PoK) breaks down because the briber can pass the PoK protocol by accessing sk intermediated by the TEE, while the TEE prevents the briber from using sk for other purposes, e.g., signing transactions. In conclusion, Dark DAO attacks facilitate large-scale vote-buying, posing a threat to DAO governance.

Defending Dark DAOs. To defend Dark DAOs, recent works consider a stronger notion, which guarantees that the prover has physical access to a secret, even in the presence of malicious provers that can use MPC or TEEs. This concept is named *proof of complete knowledge* (PoCK) by Kelkar et al. [21] or *proof of individual knowledge* (PIK) by Dziembowski et al. [13]. In Kelkar et al.’s model [21], PoCK ensures that the prover can only execute the proof by accessing a resource oracle $\mathcal{R}$, and he can “self-eavesdrop” the secret w on an unencrypted channel between the prover and the resource $\mathcal{R}$. Thus, the prover must have unencumbered knowledge of the secret. Kelkar et al. [21] introduced two schemes to realize PoCK: one based on TEEs and the other based on application-

specific integrated circuits (ASICs). The TEE-based scheme simply uses a TEE to prove unencumbered knowledge via remote attestation [28]; and the ASIC-based scheme leverages the performance gap between TEE-equipped processors and ASICs³. However, to date, not every (public) blockchain participant is equipped with a TEE or ASIC, making such solutions difficult to deploy in practice.

Thereby, we ask the following question:

Is there an alternative approach to defending against Dark DAOs that doesn't require specialized hardware and can be readily deployed on off-the-shelf systems in practice?

Our Approach. In this work, we affirmatively answer the above question by proposing a new primitive called *human-extractable zero-knowledge proofs of knowledge* (HE-ZKPoK) and designing a protocol to realize HE-ZKPoK. The main idea is inspired by human-extractable CAPTCHA puzzles [22].

CAPTCHA is an acronym for *Completely Automated Public Turing test to tell Computers and Humans Apart*. They are puzzles that are easy for humans but hard for a computer to process. CAPTCHA has several applications in cryptography [2,8,12,11,22,5], in which the human-extractable CAPTCHA proposed by Kumarasubramanian et al. [22] defines an *extraction* property of CAPTCHA. Generally speaking, it makes the following assumption. Consider an adversary receiving a CAPTCHA puzzle z_b from two random puzzles (z_0, z_1) , where the bit b is unknown to the challenger. If the adversary can solve the puzzle z_b , then the challenger can determine the bit b merely by looking at the adversary's CAPTCHA queries with the help of a human.

Intuitively, we can force the prover to solve human-extractable CAPTCHA puzzles during the protocol, along with completing a standard proof of knowledge. Then, the extractor can extract the witness bit by bit using the extraction property of human-extractable CAPTCHA. We formalize this property of HE-ZKPoK as *straight-line human-extractability*, i.e., if the prover convinces the verifier, then there exists an extractor that, with the help of a human, can extract the witness, merely by looking at the prover's CAPTCHA queries and the associated puzzles.

More specifically, a first attempt of HE-ZKPoK protocol can be as follows. Consider a binary relation $\mathcal{R}$ and a statement-witness pair $(x, w) \in \mathcal{R}$, where $w = w_1 || \dots || w_\ell$ is the binary representation of w . The prover first commits to w bit by bit and proves that $(x, w = w_1 || \dots || w_\ell) \in \mathcal{R}$ using a standard ZKPoK protocol. Then, the verifier generates 2 CAPTCHA puzzle-solution pairs $(z_0^{(i)}, s_0^{(i)})$ and $(z_1^{(i)}, s_1^{(i)})$ for each $i \in [\ell]$, and the prover selects $z_{w_i}^{(i)}$ by oblivious transfer (OT). The prover queries the human oracle to obtain the solution s_i to the puzzle $z_{w_i}^{(i)}$, $i \in [\ell]$. Finally, the prover and verifier engage in an equality test protocol to check that the prover has the puzzle solutions, where the

³Similar to Kelkar et al.'s ASIC-based scheme [21], Dziembowski et al.'s definition of PIK [13] models adversaries having slow queries to the hash oracle, which corresponds to the computational gap between TEE-equipped processors and ASICs in practice.

prover's input is the solutions $\{s_i\}_{i \in [\ell]}$, and the verifier's input is the solutions $\{s_0^{(i)}, s_1^{(i)}\}_{i \in [\ell]}$; the verifier accepts if and only if the equality test passes, i.e., $\forall i \in [\ell]$ it holds that $s_i = s_0^{(i)} \vee s_i = s_1^{(i)}$.

However, the construction described above is vulnerable to selective failure attacks. For example, the verifier could input $(z, 0^{\ell(\lambda)})$ to the OT protocol, where z is a valid puzzle. If the prover successfully completes the protocol, the verifier can infer that the corresponding bit is 0; otherwise, the bit is 1. Therefore, we need additional checks to prevent such selective failure attacks.

Contributions. Our contributions can be summarized as follows:

1. We introduce and formally define a new primitive called *human-extractable zero-knowledge proofs of knowledge* (HE-ZKPoK).
2. We propose a general construction of HE-ZKPoK based on widely used cryptographic primitives, including commitments, OT, ZKPoK, and human-extractable CAPTCHA.
3. We provide efficient instantiations of our scheme and show how HE-ZKPoK can be used to defend Dark DAO attacks in practice.

Comparing HE-ZKPoK with PoCK/PIK. A natural question is: Can we analyze our protocol in the existing PoCK/PIK framework? Unfortunately, we find that the PoCK/PIK framework is too restrictive. In the PoCK framework [21], the prover has access to a resource oracle $\mathcal{R}$ and the witness w must be transmitted over the eavesdropping channel between the prover and $\mathcal{R}$ *in plaintext*. However, it would suffice that there exists a real-world, runnable extractor that could extract the witness; requiring plaintext transmission of the witness is too restrictive. In the PIK framework [13], the adversaries are modeled to have slow queries to the hash oracle, which applies only to ASIC-based schemes. Therefore, we choose to define a new primitive HE-ZKPoK and design a protocol to realize it without requiring specialized hardware.

Related work. To protect remote electronic voting systems from coercion and vote-buying attacks, several coercion-resistant voting schemes have been proposed [20,9,1,3,25,14]. The coercion-evading strategies have two types: fake credentials [20,9,3] and re-voting [1,25,14], where fake credentials enable voters to generate fake ballots that are indistinguishable to the coercer, and re-voting allows voters to re-cast their ballots after being coerced. However, all these schemes require a basic assumption: voter authentication is inalienable [1], i.e., the coercer can neither impersonate the voter nor prevent the voter from being authenticated. Recently, the concept of Dark DAO [10,4] shows that with the help of TEEs [27,26], the coercer/vote-buyer can break inalienable authentication by creating encumbered keys, posing a potential threat to DAO governance. Faced with this situation, researchers started to consider stronger security notions; the notion of proof of complete knowledge (PoCK) was proposed by Kelkar et al. [21] and the notion of proof of individual knowledge (PIK) was proposed by Dziembowski et al. [13] concurrently. They also designed PoCK/PIK protocols based on TEEs or ASICs.

In the field of cryptography, researchers have investigated various applications of CAPTCHA puzzles [2,8,12,11,22,5]. von Ahn et al. [2] design hard AI problems for CAPTCHA, and show that the solution to these AI problems implies steganographic communication. Canetti et al. [8] uses CAPTCHA to mitigate off-line dictionary attacks on password-protected encryption. Dziembowski [12] designs a human key agreement protocol that is secure against machine-in-the-middle attacks. Diaz-Santiago and Chakraborty [11] propose an encryption scheme using CAPTCHA with security against non-human profiling adversaries. Kumara-subramanian et al. [22] explore the *extraction* property of CAPTCHA and define human-extractable CAPTCHAs. Blocki and Zhou [5] propose the notion of Proof of Human-work (PoH) and construct PoH puzzles from indistinguishability obfuscation and CAPTCHAs. Among them, the notion of human-extractable CAPTCHAs [22] connects CAPTCHAs with mainstream questions in cryptography much more closely than ever. We investigate the properties of human-extractable CAPTCHAs and build an HE-ZKPoK protocol that can effectively prevent Dark DAO attacks.

2 Preliminaries

Notations. Let $\lambda \in \mathbb{N}$ be the security parameter. We say that a function is *negligible* if it is asymptotically smaller than the inverse of any fixed polynomial. We say that an event happens with *overwhelming probability* w.r.t. λ if it happens with a probability $p(\lambda) = 1 - \epsilon(\lambda)$ where $\epsilon(\lambda)$ is a negligible function. We use the abbreviation PPT to denote *probabilistic polynomial-time*. For an NP relation $\mathcal{R}$, we use $\mathcal{L}_{\mathcal{R}}$ to denote its associated language, i.e., $\mathcal{L}_{\mathcal{R}} = \{x \mid \exists w \text{ s.t. } (x, w) \in \mathcal{R}\}$. We say that two distribution ensembles $X = \{X_{\lambda}\}_{\lambda \in \mathbb{N}}$ and $Y = \{Y_{\lambda}\}_{\lambda \in \mathbb{N}}$ are statistically (resp., computationally) indistinguishable, denoted by $X \stackrel{s}{\approx} Y$ (resp., $X \stackrel{c}{\approx} Y$), if for any computationally unbounded (resp., PPT) distinguisher D there exists a negligible function $\epsilon(\cdot)$ such that $|\Pr[D(X_{\lambda}) = 1] - \Pr[D(Y_{\lambda}) = 1]| \leq \epsilon(\lambda)$. We write $y = A(x; r)$ when the algorithm A on input x and randomness r , outputs y . We write $y \leftarrow A(x)$ for the process of picking r uniformly random and setting $y = A(x; r)$. We also write $y \leftarrow S$ for sampling y uniformly random from the set S . For an integer n , we write $[n]$ to denote the set $\{1, 2, \dots, n\}$.

2.1 CAPTCHA

CAPTCHA is an acronym for *Completely Automated Public Turing test to tell Computers and Humans Apart*, which refers to puzzles that are easy for humans but hard for automated computer programs. They are used to confirm “presence of a human” in a communication channel. In the following, we recap the formal definitions of CAPTCHA puzzles [2] and human-extractable CAPTCHA puzzles [22] (Section 2.2).

Modeling Human. The presence of a human entity is modeled as an oracle $\mathcal{H}$ that is capable of solving CAPTCHA puzzles “efficiently”. An Interactive Turing

Machine (ITM) M with oracle access to $\mathcal{H}$ is an ordinary ITM except that it has two special tapes: a write-only query tape and a read-only answer tape. When M writes a string q on its query tape, the response a from the human $\mathcal{H}$ is written on its answer tape. If q is not a valid query w.r.t. $\mathcal{H}$, a special symbol $\perp$ is written on the answer tape. Such a query-and-answer step is counted as one step in the running time of M . We use the notation $M^{\mathcal{H}}$ to denote that M has oracle access to $\mathcal{H}$.

CAPTCHA Puzzles. Let λ be the security parameter and $\ell(\cdot)$ be a fixed polynomial, we assume that the *puzzle space* P_λ and *solution space* S_λ are both discrete and finite sets such that every puzzle instance $z \in P_\lambda$ is a bit string of length $\ell(\lambda)$ and every solution $s \in S_\lambda$ is a bit string of length λ . Furthermore, we consider the function $\mathcal{H}_\lambda : P_\lambda \rightarrow S_\lambda$ that maps a puzzle instance to its corresponding solution.

With this formulation, we can view “human oracle” as computational devices which can “efficiently” compute the solution function $\mathcal{H}_\lambda$. Therefore, the algorithms that have access to “human oracle” can simply be provided with *oracle* access to the family of solution functions $\{\mathcal{H}_\lambda\}_{\lambda \in \mathbb{N}}$. Then we could define a CAPTCHA puzzle system as follows.

A CAPTCHA puzzle system is denoted as $C := (\mathcal{G}, \mathcal{H})$ where $\mathcal{G}$ is a puzzle generation algorithm and $\mathcal{H} := \{\mathcal{H}_\lambda\}_{\lambda \in \mathbb{N}}$ is a family of solution functions. The puzzle generation algorithm $\mathcal{G}$ is a PPT algorithm that takes as input the security parameter λ , and outputs a puzzle instance $z \in P_\lambda$ and its corresponding solution $s = \mathcal{H}_\lambda(z) \in S_\lambda$. The puzzle-generation process is completely *automated*, so $\mathcal{G}$ does not have access to $\mathcal{H}$.

Definition 1 (CAPTCHA Puzzles). Let λ be the security parameter, $\ell(\cdot)$ be a polynomial, $P := \{P_\lambda\}_{\lambda \in \mathbb{N}}$ and $S := \{S_\lambda\}_{\lambda \in \mathbb{N}}$ be the puzzle and solution spaces, respectively, such that $P_\lambda \subseteq \{0, 1\}^{\ell(\lambda)}$ and $S_\lambda \subseteq \{0, 1\}^\lambda$. We say that $C = (\mathcal{G}, \mathcal{H})$ is a CAPTCHA puzzle system over (P, S) if there exists a negligible function $\epsilon(\cdot)$ such that for every PPT adversary $\mathcal{A}$, we have:

$$\Pr[(z, s) \leftarrow \mathcal{G}(1^\lambda) : \mathcal{A}(1^\lambda, z) = s] \leq \epsilon(\lambda).$$

Remark 1. The security of a CAPTCHA system only holds against PPT adversaries that *do not* have oracle access to $\mathcal{H}$.

2.2 Human-Extractable CAPTCHA Puzzles

The notion of human-extractable CAPTCHA puzzles was introduced by Kumarasubramanian et al. [22]. Intuitively, if a PPT algorithm $\mathcal{A}$ can solve a CAPTCHA puzzle z generated by $\mathcal{G}$, it must make queries $q = (q_1, q_2, \dots)$ to $\mathcal{H}$, and the queries q contain sufficient information about z . More formally, suppose that z_0 and z_1 are two random and independent puzzles generated by $\mathcal{G}$, if $\mathcal{A}$ is given z_b as input and outputs the correct solution, then the queries q will contain sufficient information about z_b and no information about z_{1-b} (since z_{1-b} is independent of z_b and is never seen by $\mathcal{A}$). Therefore, taking the queries

q and puzzles z_0, z_1 as input, it should be possible with the help of a human to deduce which of the two puzzles is solved by $\mathcal{A}$. This leads to the definition of human-extractable CAPTCHA puzzles.

Definition 2 (Human-Extractable CAPTCHA Puzzles). *A CAPTCHA puzzle system $C = (\mathcal{G}, \mathcal{H})$ is said to be human-extractable if there exists an oracle PPT extractor $\mathcal{E}^{\mathcal{H}}$ and a negligible function $\epsilon(\cdot)$, such that for every oracle PPT adversary $\mathcal{A}^{\mathcal{H}}$, we have:*

$$\Pr \left[(z_0, s_0) \leftarrow \mathcal{G}(1^\lambda); (z_1, s_1) \leftarrow \mathcal{G}(1^\lambda); b \leftarrow \{0, 1\}; \right. \\ \left. (s, q, a) \leftarrow \mathcal{A}^{\mathcal{H}}(1^\lambda, z_b); b' \leftarrow \mathcal{E}^{\mathcal{H}}(1^\lambda, (z_0, z_1), q) : s = s_b \wedge b' \neq b \right] \leq \epsilon(\lambda).$$

We define the above probability as p_{fail} , i.e., the failure probability of the extractor $\mathcal{E}$ in distinguishing between the two puzzles.

3 Human-Extractable ZK Proofs of Knowledge

In this section, we formally define human-extractable zero-knowledge proofs of knowledge (HE-ZKPoK) for NP languages. Then, we present an HE-ZKPoK protocol which uses human-extractable CAPTCHA puzzles and common cryptographic primitives, including commitments, OT, and ZKPoK.

3.1 Definition

Let $C = (\mathcal{G}, \mathcal{H})$ be a human-extractable CAPTCHA puzzle system. Consider the proof system $\Pi_{\text{HE-ZKPoK}} := (\text{Prove}^{\mathcal{H}}, \text{Verify})$ consisting of a pair of PPT interactive algorithms, where $\text{Prove}^{\mathcal{H}}$ is the prover algorithm that has access to the human oracle, i.e., the CAPTCHA solution functions family $\mathcal{H}$, and Verify is the verifier algorithm. We use the following notation for $\text{Prove}^{\mathcal{H}}$ and Verify interacting on input s and t : $(\text{tr}, Q, \text{pub}) \leftarrow \langle \text{Prove}^{\mathcal{H}}(s), \text{Verify}(t) \rangle$, where tr is the transcript between the prover and verifier, $Q := (\bar{q}, \bar{a})$ are the queries to and responses from the human oracle $\mathcal{H}$, and pub is an auxiliary *public* string output by the verifier after accepting the proof at the end. We shorten the notation by writing $\langle \text{Prove}^{\mathcal{H}}(s), \text{Verify}(t) \rangle = b \in \{0, 1\}$, where $b = 0$ and $b = 1$ represent rejecting and accepting, respectively. In the following, we use $\text{view}_{\text{Verify}}$ to denote the “view” of the verifier in the interaction, consisting of its input, random coins, and the sequence of messages between the prover and the verifier.

Definition 3 (Human-Extractable ZK Proofs of Knowledge). *A protocol $\Pi_{\text{HE-ZKPoK}} = (\text{Prove}^{\mathcal{H}}, \text{Verify})$ is a HE-ZKPoK for a polynomial-time-decidable binary relation $\mathcal{R}$ over the human-extractable CAPTCHA puzzle system $C = (\mathcal{G}, \mathcal{H})$ if it has completeness, knowledge soundness, zero-knowledge, and straight-line human-extractability as defined below.*

Definition 4 (Perfect Completeness). *A protocol $\Pi_{\text{HE-ZKPoK}}$ has perfect completeness if for every pair $(x, w) \in \mathcal{R}$ we have:*

$$\Pr \left[\langle \text{Prove}^{\mathcal{H}}(x, w), \text{Verify}(x) \rangle = 1 \right] = 1.$$

Definition 5 (Knowledge Soundness). A protocol $\Pi_{\text{HE-ZKPoK}}$ has knowledge soundness if for every prover $\text{Prove}^{*\mathcal{H}}$, there exists a PPT knowledge extractor Extr that has rewindable black-box access to $\text{Prove}^{*\mathcal{H}}$ and a negligible function $\epsilon(\cdot)$, such that for any $(x, w) \in \mathcal{R}$, the extractor Extr on input x can output a valid witness w' for the statement x with overwhelming probability, i.e.,

$$\Pr \left[w' \leftarrow \text{Extr}^{(\text{Prove}^{*\mathcal{H}}(x, w), \text{Verify}(x))}(x) : \langle \text{Prove}^{*\mathcal{H}}(x, w), \text{Verify}(x) \rangle = 1 \wedge (x, w') \notin \mathcal{R} \right] \leq \epsilon(\lambda).$$

Definition 6 (Computational Zero-Knowledge). A protocol $\Pi_{\text{HE-ZKPoK}}$ has computational zero-knowledge if for every PPT verifier Verify^* , there exists a PPT simulator Sim , for every $(x, w) \in \mathcal{R}$, the simulator Sim on input x can simulate the view of the verifier Verify^* interacting with the prover $\text{Prove}^{*\mathcal{H}}$ with overwhelming probability, i.e., for any PPT verifier Verify^* , the following two distributions are computationally indistinguishable:

$$\left\{ \text{view}_{\text{Verify}^*}(\langle \text{Prove}^{*\mathcal{H}}(x, w), \text{Verify}^*(x) \rangle) \right\}_{\lambda} \stackrel{c}{\approx} \{ \text{Sim}(x) \}_{\lambda}.$$

Finally, we consider the straight-line human-extractability property for HE-ZKPoK. Informally, it means that there exists an extractor that, with the help of the human oracle, can extract the witness w from the prover's queries to $\mathcal{H}$ and the verifier's public auxiliary output. The formal definition is as follows.

Definition 7 (Straight-Line Human-Extractability). A protocol $\Pi_{\text{HE-ZKPoK}}$ has straight-line human-extractability if for every PPT $\text{Prove}^{*\mathcal{H}}$, there exists an extractor $\text{H-Extr}^{*\mathcal{H}}$ with access to the human oracle $\mathcal{H}$ and a negligible function $\epsilon(\cdot)$, such that for every pair $(x, w) \in \mathcal{R}$, we have:

$$\Pr \left[(\text{tr}, Q, \text{pub}) \leftarrow \langle \text{Prove}^{*\mathcal{H}}(x, w), \text{Verify}(x) \rangle = 1; \right. \\ \left. w' \leftarrow \text{H-Extr}^{*\mathcal{H}}(Q, \text{pub}) : (x, w') \notin \mathcal{R} \right] \leq \epsilon(\lambda).$$

Remark 2. In the above definitions, we introduce two extractors: the knowledge extractor Extr and the human extractor $\text{H-Extr}^{*\mathcal{H}}$. Although both can extract a witness, they are fundamentally different. The knowledge extractor Extr relies on rewindable black-box access to the adversarial prover $\text{Prove}^{*\mathcal{H}}$, while the human extractor $\text{H-Extr}^{*\mathcal{H}}$ requires oracle access to the human oracle $\mathcal{H}$ and is able to straight-line extract the witness.

One may argue that, with straight-line human-extractability, we do not need knowledge soundness. However, we choose to keep knowledge soundness because it provides *fall-back security*: Even if human-extractability fails in practice, HE-ZKPoK still serves as a standard ZKPoK.

3.2 Protocol Intuition

In this section, we present the intuition of our HE-ZKPoK construction. The core idea is to incorporate CAPTCHA puzzles into the execution of the zero-knowledge proof protocol. Specifically, for each bit of the witness, a 1-out-of-2

OT is performed to let the prover select a puzzle from a pair of puzzles generated by the verifier, based on the binary representation of the witness. The puzzle solutions will then be used to complete the protocol, so the prover must query the human oracle, thus ensuring human-extractability.

Cryptographic primitives. Our scheme uses a human-extractable CAPTCHA puzzle system $\text{Puzzle} = (\mathcal{G}, \mathcal{H})$ (Cf. Section 2.2) and common cryptographic primitives, including a non-interactive commitment scheme $\Pi_{\text{Com}} = (\text{KeyGen}, \text{Com}, \text{Vfy})$, a two-move OT protocol with statistical sender privacy $\Pi_{\text{OT}} = (\text{OT}_1, \text{OT}_2, \text{OT}_3)$, and a ZKPoK for all NP languages $\Pi_{\text{ZKPoK}} = (\mathcal{P}, \mathcal{V})$. Readers can refer to Appendices A, B, C for their formal definitions.

First Attempt. We now consider the construction of an HE-ZKPoK protocol $\Pi_{\text{HE-ZKPoK}} = (\text{Prove}^{\mathcal{H}}, \text{Verify})$ utilizing the above primitives. The prover Prove in our protocol has oracle access to the human oracle $\mathcal{H}$.

First, let $w = w_1 || \dots || w_\ell$ be the binary representation of w . The prover commits to w bit by bit and proves that $(x, w = w_1 || \dots || w_\ell) \in \mathcal{R}$ using a standard ZKPoK protocol.

Then, for each bit of the witness w , the verifier generates 2 instances of CAPTCHA puzzle-solution pairs independently, resulting in a total of 2ℓ pairs for $|w| = \ell$. That is, for $i \in [\ell]$, the verifier computes $(z_0^{(i)}, s_0^{(i)}) \leftarrow \mathcal{G}(1^\lambda)$ and $(z_1^{(i)}, s_1^{(i)}) \leftarrow \mathcal{G}(1^\lambda)$. The prover selects ℓ puzzles $\{z_{w_i}^{(i)}\}_{i \in [\ell]}$ using ℓ OTs in parallel, according to the witness w . After receiving the ℓ puzzles, the prover queries the human oracle $\mathcal{H}$ to obtain ℓ solutions $\{s_i\}_{i \in [\ell]}$ to the puzzles.

Next, the prover and the verifier engage in an *equality test protocol* to test that the prover indeed has a puzzle solution. The prover takes as input the solution $\{s_i\}_{i \in [\ell]}$ and the verifier takes as input the two solutions $\{s_0^{(i)}, s_1^{(i)}\}_{i \in [\ell]}$; the verifier obtains $b_i := (s_i = s_0^{(i)}) \vee (s_i = s_1^{(i)})$ and the prover obtains nothing. The equality test is realized by secure two-party computation (Cf. Appendix D).

Finally, the verifier accepts if and only if the equality test passes, i.e., $b_i = 1$ for $i \in [\ell]$. If the verifier accepts, he will publish all the puzzles $\{z_0^{(i)}, z_1^{(i)}\}_{i \in [\ell]}$ as the public auxiliary strings.

Vulnerability. Unfortunately, the above construction is vulnerable to the following selective failure attacks:

- During the OT execution, to probe the first bit w_1 of the witness w , the verifier generates a valid puzzle z and sets $z_0^{(1)} = z$ and $z_1^{(1)} = 0^{\ell(\lambda)}$ as the OT inputs. If the prover successfully completes the protocol, then $w_1 = 0$; if the prover aborts, the malicious verifier infers that $w_1 = 1$.
- During the equality test, to probe the first bit w_1 of the witness w , the verifier sets $(s_0^{(1)}, r)$ as the inputs to the equality test, where r is random. If the equality test outputs 1, then $w_1 = 0$; if the equality test outputs 0, the malicious verifier infers that $w_1 = 1$.

Therefore, the verifier needs to generate another zero-knowledge proof to prove the well-formedness of his OT inputs, and we need to modify the equality test functionality so that the verifier always obtains 0 if his inputs are invalid.

In addition, the prover also needs to prove that he selects $z_{w_i}^{(i)}$ according to the committed bit w_i . We present the final construction of the HE-ZKPoK protocol in the following section.

3.3 Protocol Description

Our HE-ZKPoK protocol follows the above idea, with additional zero-knowledge proofs during OT executions and modifications to the equality test functionality. Concretely, the HE-ZKPoK verifier proves that the inputs to OT are well-formed, and the HE-ZKPoK prover proves that he selects the corresponding puzzle according to the committed bit w_i ; in the equality test, the verifier always obtains 0 if he provides invalid inputs.

A subtle point is that, to ensure that the human extractor successfully extracts the witness, the other puzzle must be statistically hidden from the prover. Therefore, the OT must provide statistical sender privacy, and the equality test must provide statistical security for the verifier.

The modified equality test is as follows. We use Π_{eq} to denote the secure two-party computation protocol with statistical security for the verifier (see Appendix D for formal definition) for functionality $f = (f_1, f_2)$ defined in Fig. 1.

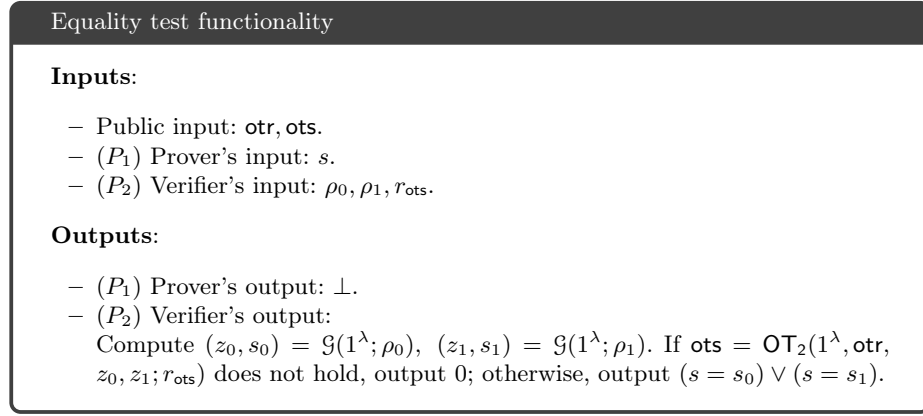


Fig. 1: Equality test functionality

The equality test functionality takes the OT transcript otr, ots as public input. The prover's input is solution s , and the verifier's input is randomness $\rho_0, \rho_1, r_{\text{ots}}$, where ρ_0, ρ_1 are used for generating puzzle-solution pairs, and r_{ots} is used in OT. If the verifier's input is invalid, he always obtains 0; otherwise, his output is the equality test result $(s = s_0) \vee (s = s_1)$.

Finally, the protocol description of $\Pi_{\text{HE-ZKPoK}}$ is shown in Fig. 2, and we formalize the relations $\mathcal{R}_1^*, \mathcal{R}_2^*, \mathcal{R}_3^*$ of the ZKP/ZKPoK protocols in Fig. 3.

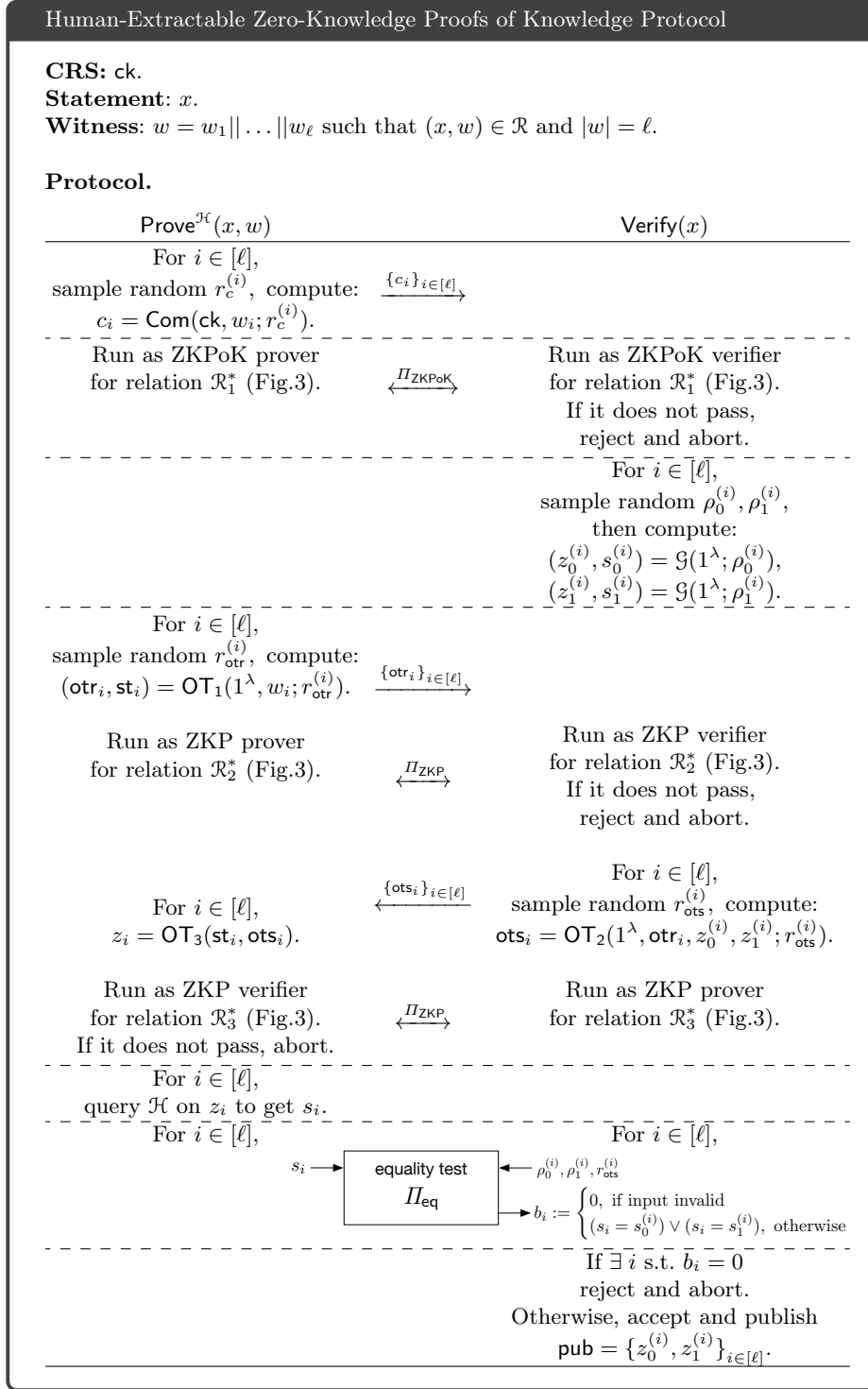
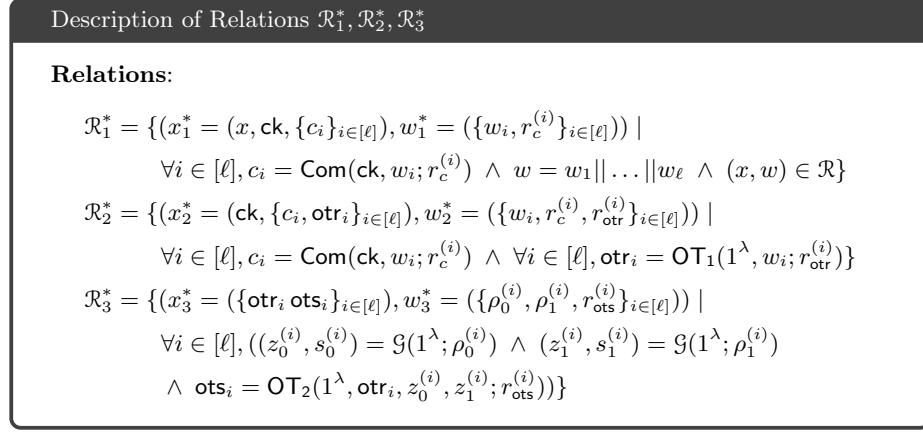


Fig. 2: Human-Extractable Zero-Knowledge Proofs of Knowledge Protocol.

Fig. 3: Description of Relations $\mathcal{R}_1^*, \mathcal{R}_2^*, \mathcal{R}_3^*$.

Theorem 1. Assume that $\Pi_{\text{Com}} = (\text{KeyGen}, \text{Com}, \text{Vfy})$ is a non-interactive commitment scheme, $\Pi_{\text{OT}} = (\text{OT}_1, \text{OT}_2, \text{OT}_3)$ is a two-move OT protocol with statistical sender privacy, $\Pi_{\text{ZKP}} = (\mathcal{P}_{\text{zk}}, \mathcal{V}_{\text{zk}})$ is a ZKP protocol, $\Pi_{\text{ZKPoK}} = (\mathcal{P}, \mathcal{V})$ is a ZKPoK protocol, Π_{eq} is a secure two-party computation protocol with statistical security for the verifier, and $\text{Puzzle} = (\mathcal{G}, \mathcal{H})$ is a human-extractable CAPTCHA puzzle system. Then, the protocol depicted in Fig. 2 is an HE-ZKPoK with perfect completeness, knowledge soundness, computational zero-knowledge, and straight-line human-extractability.

Proof sketch. (Perfect Completeness) Perfect completeness is straightforward. It follows from the perfect completeness of $\Pi_{\text{ZKPoK}}, \Pi_{\text{ZKP}}$, and perfect correctness of $\Pi_{\text{OT}}, \Pi_{\text{eq}}$.

(Knowledge Soundness) Knowledge soundness directly follows from the knowledge soundness of the underlying ZKPoK protocol Π_{ZKPoK} .

(Zero-Knowledge) To prove the zero-knowledge property, given a PPT verifier Verify^* , we need to construct a PPT simulator Sim that on input x could simulate the view of the verifier Verify^* interacting with a real prover $\text{Prove}^{\mathcal{H}}$. The simulator Sim performs as follows.

1. **Commitment:** First, Sim generates ℓ commitments of 0 and sends them to the verifier Verify^* .
2. **Simulate ZKP:** For the ZKP/ZKPoK that Sim runs as the prover, Sim invokes the simulator of the underlying ZKP/ZKPoK protocols to simulate the proofs.
3. **OT:** For all the ℓ OTs, Sim uses 0 as input and follows the protocol to complete the OT executions.
4. **Check ZKP:** Sim checks Verify^* 's ZKP for relation $\mathcal{R}_3^*$. If it does not pass, Sim aborts.

5. **Equality Test:** Sim invokes the simulator of the secure two-party computation Π_{eq} to simulate Verify^* 's view during the equality test.

By the hiding property of Π_{Com} , zero-knowledge property of Π_{ZKP} , Π_{ZKPoK} , receiver privacy of Π_{OT} , security of Π_{eq} , and soundness of Π_{ZKP} , the simulated view and real view are computationally indistinguishable.

(*Straight-Line Human-Extractability*) To prove that the protocol is straight-line human-extractable, we need to construct an extractor $\text{H-Extr}^{\mathcal{H}}$ with access to human oracle $\mathcal{H}$ that on inputs $(Q = (\bar{q}, \bar{a}), \text{pub})$ can extract a witness w with overwhelming probability if the prover convinces the verifier.

The extractor $\text{H-Extr}^{\mathcal{H}}$ runs the human extractor $\mathcal{E}$ of **Puzzle** internally, and the extractor $\text{H-Extr}^{\mathcal{H}}$ could forward the queries from the extractor $\mathcal{E}$ to the human oracle $\mathcal{H}$, and return the responses. It performs as follows:

Human Extractor $\text{H-Extr}^{\mathcal{H}}(\text{pub}, Q)$:

1. The extractor $\text{H-Extr}^{\mathcal{H}}$ first parses $\text{pub} = \{z_0^{(i)}, z_1^{(i)}\}_{i \in [\ell]}$ and $Q = (\bar{q}, \bar{a})$.
2. For all $i \in [\ell]$, the extractor $\text{H-Extr}^{\mathcal{H}}$ runs the human extractor $\mathcal{E}^{\mathcal{H}}$ on input $(1^\lambda, (z_0^{(i)}, z_1^{(i)}), \bar{q})$ and obtains b_i , i.e., $b_i \leftarrow \mathcal{E}^{\mathcal{H}}(1^\lambda, (z_0^{(i)}, z_1^{(i)}), \bar{q})$.
3. The extractor $\text{H-Extr}^{\mathcal{H}}$ outputs the concatenated string $b_1 || \dots || b_\ell$.

Then we argue that the output string $b_1 || \dots || b_\ell$ of the extractor $\text{H-Extr}^{\mathcal{H}}$ is a valid witness for the given statement x except with negligible probability. We first argue that if the extraction is successful, then $b_1 || \dots || b_\ell$ is a valid witness. Then, we show that the extraction failure probability is negligible.

Witness Validity. First, the prover proves that the committed bits are a valid witness for x by the standard ZKPoK. During OT executions, the prover proves that he selects the corresponding puzzle according to the committed bits, and the other puzzle is hidden from the prover. Thus, by the soundness of Π_{ZKP} , Π_{ZKPoK} , the extracted value $b_1 || \dots || b_\ell$ is a valid witness, if the extraction is successful.

Extraction Failure Probability. Let $\{w_j\}_{j \in [\ell]}$ be the committed bits. We say that $\text{H-Extr}^{\mathcal{H}}(1^\lambda, \bar{q}, \text{pub})$ *fails at step j* if $b_j \neq w_j$ and we define the probability $p_j(\lambda)$ that the extractor $\text{H-Extr}^{\mathcal{H}}$ fails at step j as

$$\Pr \left[\begin{array}{l} (x, w) \leftarrow \mathcal{A}(1^\lambda); (\text{tr}, Q, \text{pub}) \leftarrow \langle \text{Prove}^{*\mathcal{H}}(x, w), \text{Verify}(x) \rangle = 1; \\ (\bar{q}, \bar{a}) \leftarrow Q; \{z_0^{(i)}, z_1^{(i)}\}_{i \in [\ell]} \leftarrow \text{pub}; b_j \leftarrow \mathcal{E}^{\mathcal{H}}(1^\lambda, \{z_0^{(j)}, z_1^{(j)}\}, \bar{q}) : \\ w = w_1 || \dots || w_\ell \wedge b_j \neq w_j \end{array} \right].$$

If we prove that $p_j(\lambda)$ is negligible for every $j \in [\ell]$, then by union bound, it follows that the output of $\text{H-Extr}^{\mathcal{H}}$ is always equal to the prover's witness w except for negligible probability. To show that $p_j(\lambda)$ is negligible, it suffices to show that $p_j(\lambda)$ and p_{fail} in Definition 2 are negligibly close.

Informally, the proof follows from the fact that the prover $\text{Prove}^{\mathcal{H}}$ solves one CAPTCHA puzzle $z_{w_j}^{(j)}$ and passes the equality test using the solution $s_{w_j}^{(j)}$, and the other CAPTCHA puzzle $z_{1-w_j}^{(j)}$ is statistically hidden from the prover's

view (that’s why OT must have statistical sender privacy and Π_{eq} must have statistical security for the verifier). Through several hybrid experiments, we can simulate the verifier using only $z_{w_j}^{(j)}, s_{w_j}^{(j)}$ during the protocol, so $p_j(\lambda)$ and p_{fail} are negligibly close by the statistical sender privacy of Π_{OT} , statistical security for verifier in Π_{eq} , and statistical zero-knowledge of Π_{ZKP} for relation $\mathcal{R}_3^*$. $\square$

4 Instantiation and Discussion

4.1 Instantiation

Non-interactive Commitment. The non-interactive commitment scheme can be instantiated using any common commitment scheme, such as Pedersen commitment or hash-based commitment.

Two-move OT with Statistical Sender Privacy. The two-move statistical sender secure OT can be instantiated using the protocol in [18], which develops two novel OT protocols based on the Quadratic Residuosity Assumption and the N’tth Residuosity Assumption, respectively, both of which provide statistical sender privacy.

Zero-Knowledge Proofs of Knowledge. The zero-knowledge proofs of knowledge can be instantiated using the protocol in [16], which employs a statistically binding commitment scheme to achieve statistical soundness and computational zero-knowledge for all NP languages in the plain model. In practice, it is also possible to use more efficient zero-knowledge arguments of knowledge (e.g., bulletproofs [6], Groth16 [17]), with additional computational assumptions.

Secure Two-Party Computation with Statistical Security for Verifier. The secure two-party computation with statistical security for verifier can be instantiated using Yao’s garbled circuit [30] and the cut-and-choose technique [24]. With the verifier playing the circuit evaluator, choosing OT with statistical receiver privacy, and using statistically binding commitment, we can achieve verifier’s statistical security.

4.2 Blockchain Deployment

In the blockchain context, authentication typically requires a user to remotely prove knowledge of a secret key, which can be formalized by the relation $\mathcal{R} = \{(\text{pk}, \text{sk}) \mid \text{pk} = g^{\text{sk}}\}$. The authentication procedure is performed via a HE-ZKPoK protocol between the voter (prover) and the authentication server (verifier). The authentication server could be instantiated with a smart contract; another choice is to instantiate it with a TEE, which is more efficient. After authentication, the authentication server posts the corresponding auxiliary strings (i.e., the CAPTCHA puzzles) on the blockchain.

When the voter is honest, he (a human entity) executes the HE-ZKPoK protocol and solves the CAPTCHA puzzles himself. Only the voter learns the CAPTCHA

queries during the execution, ensuring that the secret key sk remains confidential. Again, we emphasize that the HE-ZKPoK protocol proceeds without relying on any hardware on the prover side.

If a briber attempts to launch a Dark DAO attack [4], it will be infeasible. Suppose the briber creates an encumbered secret key sk inside a TEE. Then, during the HE-ZKPoK execution, the TEE cannot solve the CAPTCHA puzzles itself, so the CAPTCHA queries must be forwarded to the human oracle, which can be under the briber’s control. This forwarding exposes the CAPTCHA queries to the briber, enabling him to run the human extractor and recover the secret key sk . Therefore, HE-ZKPoK prevents Dark DAO attacks effectively.

4.3 Comparing with the result of [22]

In the paper that proposes human-extractable CAPTCHA puzzles [22], the authors also build a UC puzzle [23] using CAPTCHA. Following the result of [23], this can be used to realize general UC-secure MPC and thus implies ZK. However, we highlight the key differences between [22]’s result and our work:

- Our work provides significant practical motivation: Dark DAO attacks pose a threat to DAO governance, and HE-ZKPoK can effectively prevent them; whereas [22] presents it as a theoretical exploration and mainly considers general feasible results.
- Our construction is more efficient and practical. In the work of [22], UC puzzles need to be combined with strong non-malleable witness-indistinguishable (SNMWI) protocols [23] to realize general MPC, which is a great theoretical result, but hard to deploy in practice. In contrast, our protocol constructs HE-ZKPoK directly, which is much more efficient and practical.

4.4 Empirical Evaluation

In the practical execution of the HE-ZKPoK protocol, the running time cost is dominated by the CAPTCHA-solving process. According to a recent empirical study by Searles et al. [29], it takes a human 3.1–4.9 seconds on average to solve a single reCAPTCHA. Therefore, if a witness consists of 128 bits, the user would need to solve 128 reCAPTCHAs, resulting in a total solving time of approximately 396.8–627.2 seconds.

5 Conclusion

In this work, we propose a new primitive called human-extractable zero-knowledge proofs of knowledge (HE-ZKPoK) that can effectively prevent Dark DAOs. We then present an HE-ZKPoK protocol based on human-extractable CAPTCHA puzzles and common cryptographic primitives, achieving the first Dark DAO defense protocol without specialized hardware. We further discuss the instantiation, blockchain deployment, and empirical cost, providing a potential solution to improve DAO governance.

References

1. Achenbach, D., Kempka, C., Löwe, B., Müller-Quade, J.: Improved coercion-resistant electronic elections through deniable re-voting. *USENIX Journal of Election Technology and Systems* pp. 26–45 (2015)
2. von Ahn, L., Blum, M., Hopper, N.J., Langford, J.: CAPTCHA: Using hard AI problems for security. In: Biham, E. (ed.) *Advances in Cryptology – EUROCRYPT 2003. Lecture Notes in Computer Science*, vol. 2656, pp. 294–311. Springer Berlin Heidelberg, Germany, Warsaw, Poland (May 4–8, 2003). https://doi.org/10.1007/3-540-39200-9_18
3. Araújo, R., Barki, A., Brunet, S., Traoré, J.: Remote electronic voting can be efficient, verifiable and coercion-resistant. In: Clark, J., Meiklejohn, S., Ryan, P.Y.A., Wallach, D.S., Brenner, M., Rohloff, K. (eds.) *FC 2016 Workshops. Lecture Notes in Computer Science*, vol. 9604, pp. 224–232. Springer Berlin Heidelberg, Germany, Christ Church, Barbados (Feb 26, 2016). https://doi.org/10.1007/978-3-662-53357-4_15
4. Austgen, J., Fábrega, A., Allen, S., Babel, K., Kelkar, M., Juels, A.: DAO decentralization: Voting-bloc entropy, bribery, and dark daos. *CoRR* **abs/2311.03530** (2023). <https://doi.org/10.48550/ARXIV.2311.03530>
5. Blocki, J., Zhou, H.S.: Designing proof of human-work puzzles for cryptocurrency and beyond. In: Hirt, M., Smith, A.D. (eds.) *TCC 2016-B: 14th Theory of Cryptography Conference, Part II. Lecture Notes in Computer Science*, vol. 9986, pp. 517–546. Springer Berlin Heidelberg, Germany, Beijing, China (Oct 31 – Nov 3, 2016). https://doi.org/10.1007/978-3-662-53644-5_20
6. Bünz, B., Bootle, J., Boneh, D., Poelstra, A., Wuille, P., Maxwell, G.: Bulletproofs: Short proofs for confidential transactions and more. In: *2018 IEEE Symposium on Security and Privacy*. pp. 315–334. IEEE Computer Society Press, San Francisco, CA, USA (May 21–23, 2018). <https://doi.org/10.1109/SP.2018.00020>
7. Buterin, V.: Bootstrapping a decentralized autonomous corporation: part i. *Bitcoin Magazine* (2013), <https://bitcoinmagazine.com/technical/bootstrapping-a-decentralized-autonomous-corporation-part-i-1379644274>, online; accessed 21 Jan. 2026
8. Canetti, R., Halevi, S., Steiner, M.: Mitigating dictionary attacks on password-protected local storage. In: Dwork, C. (ed.) *Advances in Cryptology – CRYPTO 2006. Lecture Notes in Computer Science*, vol. 4117, pp. 160–179. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 20–24, 2006). https://doi.org/10.1007/11818175_10
9. Clarkson, M.R., Chong, S., Myers, A.C.: Civitas: Toward a secure voting system. In: *2008 IEEE Symposium on Security and Privacy*. pp. 354–368. IEEE Computer Society Press, Oakland, CA, USA (May 18–21, 2008). <https://doi.org/10.1109/SP.2008.32>
10. Daian, P., Kell, T., Miers, I., Juels, A.: On-chain vote buying and the rise of dark daos (2018), <https://initc3org.medium.com/on-chain-vote-buying-and-the-rise-of-dark-daos-b01f5bd77030>, online; accessed 21 Jan. 2026
11. Diaz-Santiago, S., Chakraborty, D.: On securing communication from profilers. In: Samarati, P., Lou, W., Zhou, J. (eds.) *SECRYPT 2012 - Proceedings of the International Conference on Security and Cryptography, Rome, Italy, 24–27 July, 2012, SECRYPT is part of ICETE - The International Joint Conference on e-Business and Telecommunications*. pp. 154–162. SciTePress (2012)

12. Dziembowski, S.: How to pair with a human. In: Garay, J.A., De Prisco, R. (eds.) SCN 10: 7th International Conference on Security in Communication Networks. Lecture Notes in Computer Science, vol. 6280, pp. 200–218. Springer Berlin Heidelberg, Germany, Amalfi, Italy (Sep 13–15, 2010). https://doi.org/10.1007/978-3-642-15317-4_14
13. Dziembowski, S., Faust, S., Lizej, T.: Individual cryptography. In: Handschuh, H., Lysyanskaya, A. (eds.) Advances in Cryptology – CRYPTO 2023, Part II. Lecture Notes in Computer Science, vol. 14082, pp. 547–579. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 20–24, 2023). https://doi.org/10.1007/978-3-031-38545-2_18
14. Giustolisi, R., Garjan, M.S., Schürmann, C.: Thwarting last-minute voter coercion. In: 2024 IEEE Symposium on Security and Privacy. pp. 3423–3439. IEEE Computer Society Press, San Francisco, CA, USA (May 19–23, 2024). <https://doi.org/10.1109/SP54263.2024.00112>
15. Goldreich, O.: The Foundations of Cryptography - Volume 2: Basic Applications. Cambridge University Press (2004). <https://doi.org/10.1017/CBO9780511721656>, <http://www.wisdom.weizmann.ac.il/%7Eoded/foc-vol2.html>
16. Goldreich, O., Micali, S., Wigderson, A.: Proofs that yield nothing but their validity or all languages in NP have zero-knowledge proof systems. *Journal of the ACM* **38**(3), 691–729 (Jul 1991). <https://doi.org/10.1145/116825.116852>
17. Groth, J.: On the size of pairing-based non-interactive arguments. In: Fischlin, M., Coron, J.S. (eds.) Advances in Cryptology – EUROCRYPT 2016, Part II. Lecture Notes in Computer Science, vol. 9666, pp. 305–326. Springer Berlin Heidelberg, Germany, Vienna, Austria (May 8–12, 2016). https://doi.org/10.1007/978-3-662-49896-5_11
18. Halevi, S., Kalai, Y.T.: Smooth projective hashing and two-message oblivious transfer. *Journal of Cryptology* **25**(1), 158–193 (Jan 2012). <https://doi.org/10.1007/s00145-010-9092-8>
19. Hassan, S., Filippi, P.D.: Decentralized autonomous organization. *Internet Policy Rev.* **10**(2) (2021). <https://doi.org/10.14763/2021.2.1556>
20. Juels, A., Catalano, D., Jakobsson, M.: Coercion-resistant electronic elections. In: Proceedings of the 2005 ACM Workshop on Privacy in the Electronic Society. pp. 61–70 (2005)
21. Kelkar, M., Babel, K., Daian, P., Austgen, J., Buterin, V., Juels, A.: Complete knowledge: Preventing encumbrance of cryptographic secrets. In: Luo, B., Liao, X., Xu, J., Kirda, E., Lie, D. (eds.) ACM CCS 2024: 31st Conference on Computer and Communications Security. pp. 2415–2429. ACM Press, Salt Lake City, UT, USA (Oct 14–18, 2024). <https://doi.org/10.1145/3658644.3690273>
22. Kumarasubramanian, A., Ostrovsky, R., Pandey, O., Wadia, A.: Cryptography using captcha puzzles. In: Kurosawa, K., Hanaoka, G. (eds.) PKC 2013: 16th International Conference on Theory and Practice of Public Key Cryptography. Lecture Notes in Computer Science, vol. 7778, pp. 89–106. Springer Berlin Heidelberg, Germany, Nara, Japan (Feb 26 – Mar 1, 2013). https://doi.org/10.1007/978-3-642-36362-7_7
23. Lin, H., Pass, R., Venkatasubramanian, M.: A unified framework for concurrent security: universal composability from stand-alone non-malleability. In: Mitzenmacher, M. (ed.) 41st Annual ACM Symposium on Theory of Computing. pp. 179–188. ACM Press, Bethesda, MD, USA (May 31 – Jun 2, 2009). <https://doi.org/10.1145/1536414.1536441>

24. Lindell, Y., Pinkas, B.: An efficient protocol for secure two-party computation in the presence of malicious adversaries. In: Naor, M. (ed.) *Advances in Cryptology – EUROCRYPT 2007*. Lecture Notes in Computer Science, vol. 4515, pp. 52–78. Springer Berlin Heidelberg, Germany, Barcelona, Spain (May 20–24, 2007). https://doi.org/10.1007/978-3-540-72540-4_4
25. Lueks, W., Querejeta-Azurmendi, I., Troncoso, C.: VoteAgain: A scalable coercion-resistant voting system. In: Capkun, S., Roesner, F. (eds.) *USENIX Security 2020: 29th USENIX Security Symposium*. pp. 1553–1570. USENIX Association (Aug 12–14, 2020), <https://www.usenix.org/conference/usenixsecurity20/presentation/lueks>
26. McKeen, F., Alexandrovich, I., Anati, I., Caspi, D., Johnson, S., Leslie-Hurd, R., Rozas, C.V.: Intel® software guard extensions (intel® SGX) support for dynamic memory management inside an enclave. In: *Proceedings of the Hardware and Architectural Support for Security and Privacy 2016, HASP@ICSA 2016*, Seoul, Republic of Korea, June 18, 2016. pp. 10:1–10:9. ACM (2016). <https://doi.org/10.1145/2948618.2954331>
27. McKeen, F., Alexandrovich, I., Berenzon, A., Rozas, C.V., Shafi, H., Shanbhogue, V., Savagaonkar, U.R.: Innovative instructions and software model for isolated execution. In: Lee, R.B., Shi, W. (eds.) *HASP 2013, The Second Workshop on Hardware and Architectural Support for Security and Privacy*, Tel-Aviv, Israel, June 23–24, 2013. p. 10. ACM (2013). <https://doi.org/10.1145/2487726.2488368>
28. Pass, R., Shi, E., Tramèr, F.: Formal abstractions for attested execution secure processors. In: Coron, J.S., Nielsen, J.B. (eds.) *Advances in Cryptology – EUROCRYPT 2017, Part I*. Lecture Notes in Computer Science, vol. 10210, pp. 260–289. Springer, Cham, Switzerland, Paris, France (Apr 30 – May 4, 2017). https://doi.org/10.1007/978-3-319-56620-7_10
29. Searles, A., Nakatsuka, Y., Ozturk, E., Paverd, A., Tsudik, G., Enkoji, A.: An empirical study & evaluation of modern CAPTCHAs. In: Calandrino, J.A., Troncoso, C. (eds.) *USENIX Security 2023: 32nd USENIX Security Symposium*. pp. 3081–3097. USENIX Association, Anaheim, CA, USA (Aug 9–11, 2023), <https://www.usenix.org/conference/usenixsecurity23/presentation/searles>
30. Yao, A.C.C.: How to generate and exchange secrets (extended abstract). In: *27th Annual Symposium on Foundations of Computer Science*. pp. 162–167. IEEE Computer Society Press, Toronto, Ontario, Canada (Oct 27–29, 1986). <https://doi.org/10.1109/SFCS.1986.25>

A Non-interactive Commitment Scheme

A non-interactive commitment scheme allows a party to commit to a message m by publishing a commitment c , which can be opened later. Two important security properties of commitment schemes are hiding and binding. Informally, the hiding property means that the receiver cannot learn anything about the committed message m from the commitment c , and the binding property means that the sender cannot open the commitment to a different message m' . The formal definition is as follows.

Definition 8 (Non-interactive Commitment). *Consider the message space $\mathcal{M}_\lambda$, commitment space $\mathcal{C}_\lambda$ and decommitment space $\mathcal{D}_\lambda$, a non-interactive commitment scheme Π_{Com} consists of three PPT algorithms :*

1. $\text{ck} \leftarrow \text{KeyGen}(1^\lambda)$ is the key generation algorithm that takes inputs as the security parameter 1^λ , and outputs the commitment key ck .
2. $(c, d) \leftarrow \text{Com}(\text{ck}, m)$ is the commitment algorithm that takes input as the commitment key ck and a message $m \in \mathcal{M}_\lambda$, and outputs the commitment $c \in \mathcal{C}_\lambda$ and the decommitment value $d \in \mathcal{D}_\lambda$.
3. $1/0 \leftarrow \text{Vfy}(\text{ck}, c, m, d)$ is the verification algorithm that takes input as the commitment key ck , the commitment c , the message m , and the decommitment value d , and outputs 1 or 0 indicating acceptance or rejection.

A non-interactive commitment scheme $\Pi_{\text{Com}} := (\text{KeyGen}, \text{Com}, \text{Vfy})$ satisfies the binding and hiding properties as defined below.

Definition 9 (Binding). We say that Π_{Com} is binding if for any PPT adversary $\mathcal{A}$, there exists a negligible function $\epsilon(\cdot)$ such that:

$$\Pr \left[\begin{array}{l} \text{ck} \leftarrow \text{KeyGen}(1^\lambda); (c, m_0, d_0, m_1, d_1) \leftarrow \mathcal{A}(\text{ck}) : \\ m_0 \neq m_1 \wedge \text{Vfy}(\text{ck}, c, m_0, d_0) = 1 \wedge \text{Vfy}(\text{ck}, c, m_1, d_1) = 1 \end{array} \right] \leq \epsilon(\lambda).$$

Definition 10 (Hiding). We say that Π_{Com} is hiding if for any PPT adversary $\mathcal{A}$, there exists a negligible function $\epsilon(\cdot)$ such that:

$$\left| \Pr \left[\begin{array}{l} \text{ck} \leftarrow \text{KeyGen}(1^\lambda); (m_0, m_1, \text{st}) \leftarrow \mathcal{A}(\text{ck}); b \leftarrow \{0, 1\}; \\ (c, d) \leftarrow \text{Com}(\text{ck}, m_b); b' \leftarrow \mathcal{A}(c, \text{st}) : b = b' \end{array} \right] - \frac{1}{2} \right| \leq \epsilon(\lambda).$$

B Two-move OT with Statistical Sender Privacy

Below, we present the formal definition of a two-move OT protocol with statistical sender privacy.

Definition 11 (Two-move OT). Let λ be the security parameter and $\ell(\cdot)$ be a polynomial. A two-move OT protocol Π_{OT} consists of three PPT algorithms:

1. $(\text{otr}, \text{st}) \leftarrow \text{OT}_1(1^\lambda, c)$ is the receiver's message algorithm that takes the security parameter λ and a choice bit c as input, and it outputs the receiver's message otr and an internal state st .
2. $\text{ots} \leftarrow \text{OT}_2(1^\lambda, \text{otr}, m_0, m_1)$ is the sender's message algorithm that takes the security parameter λ , the receiver's message otr , and two strings $m_0, m_1 \in \{0, 1\}^{\ell(\lambda)}$ as input, and it outputs the sender's message ots .
3. $m_c \leftarrow \text{OT}_3(\text{st}, \text{ots})$ is the receiver's output algorithm that takes st and the sender's message ots as input, and it outputs m_c .

Definition 12 (Computational Receiver Privacy). We say that Π_{OT} has computational receiver privacy if for any PPT adversary $\mathcal{A}$, there exists a negligible function $\epsilon(\cdot)$ such that:

$$\left| \Pr \left[\begin{array}{l} (\text{otr}, \text{st}) \leftarrow \text{OT}_1(1^\lambda, 0) : \\ \mathcal{A}(1^\lambda, \text{otr}) = 1 \end{array} \right] - \Pr \left[\begin{array}{l} (\text{otr}, \text{st}) \leftarrow \text{OT}_1(1^\lambda, 1) : \\ \mathcal{A}(1^\lambda, \text{otr}) = 1 \end{array} \right] \right| \leq \epsilon(\lambda).$$

Definition 13 (Statistical Sender Privacy). *We say that Π_{OT} has statistical sender privacy if for any adversary $\mathcal{A}$, there exists a negligible function $\epsilon(\cdot)$ such that for any $m_0, m_1 \in \{0, 1\}^{\ell(\lambda)}$, we have:*

$$\left| \Pr \left[\begin{array}{l} (\text{otr}, \text{st}) \leftarrow \text{OT}_1(1^\lambda, 0); \\ \text{ots} \leftarrow \text{OT}_2(1^\lambda, \text{otr}, m_0, m_1); \\ \mathcal{A}(1^\lambda, \text{ots}) = 1 \end{array} \right] - \Pr \left[\begin{array}{l} (\text{otr}, \text{st}) \leftarrow \text{OT}_1(1^\lambda, 0); \\ \text{ots} \leftarrow \text{OT}_2(1^\lambda, \text{otr}, m_0, 0^{\ell(\lambda)}); \\ \mathcal{A}(1^\lambda, \text{ots}) = 1 \end{array} \right] \right| \leq \epsilon(\lambda)$$

and

$$\left| \Pr \left[\begin{array}{l} (\text{otr}, \text{st}) \leftarrow \text{OT}_1(1^\lambda, 1); \\ \text{ots} \leftarrow \text{OT}_2(1^\lambda, \text{otr}, m_0, m_1); \\ \mathcal{A}(1^\lambda, \text{ots}) = 1 \end{array} \right] - \Pr \left[\begin{array}{l} (\text{otr}, \text{st}) \leftarrow \text{OT}_1(1^\lambda, 1); \\ \text{ots} \leftarrow \text{OT}_2(1^\lambda, \text{otr}, 0^{\ell(\lambda)}, m_1); \\ \mathcal{A}(1^\lambda, \text{ots}) = 1 \end{array} \right] \right| \leq \epsilon(\lambda).$$

C Zero-Knowledge Proofs

Consider a polynomial-time-decidable binary relation $\mathcal{R}$, for pair $(x, w) \in \mathcal{R}$, we call x the statement and w the witness. In a zero-knowledge proof (ZKP) protocol, the prover convinces a verifier that a statement x is true, while the verifier learns nothing except the validity of the assertion. In a zero-knowledge proof of knowledge (ZKPoK) protocol, the prover needs to prove that he possesses some witness w for a statement x such that $(x, w) \in \mathcal{R}$,

Formally, a ZKP protocol for a binary relation $\mathcal{R}$ consists of a pair of PPT interactive algorithms $\Pi_{\text{ZK}} := (\mathcal{P}, \mathcal{V})$ called the prover and verifier. We use the following notation for $\mathcal{P}$ and $\mathcal{V}$ interacting on input s and t : $\text{tr} \leftarrow \langle \mathcal{P}(s), \mathcal{V}(t) \rangle$, where tr is the transcript between the prover and verifier. We shorten the notation by writing $\langle \mathcal{P}(s), \mathcal{V}(t) \rangle = b \in \{0, 1\}$, where $b = 0$ and $b = 1$ correspond to the verifier rejecting and accepting, respectively. In the following, we use $\text{view}_{\mathcal{V}}$ to denote the “view” of the verifier $\mathcal{V}$ in the interaction, consisting of its input, its random coins used in the protocol, and the sequence of messages between the prover and the verifier.

We say that Π_{ZK} is a ZKP for $\mathcal{R}$ if it has the properties of completeness, soundness, and zero-knowledge. For ZKPoK, we additionally require the property of knowledge soundness as defined below.

Definition 14 (Perfect Completeness). *We say that Π_{ZK} is perfectly complete if for every pair $(x, w) \in \mathcal{R}$ we have:*

$$\Pr [\langle \mathcal{P}(x, w), \mathcal{V}(x) \rangle = 1] = 1.$$

Definition 15 (Statistical Soundness). *We say that Π_{ZK} is statistically sound if for every prover $\mathcal{P}^*$, there exists a negligible function $\epsilon(\cdot)$ such that for every $x \notin \mathcal{L}_{\mathcal{R}}$, we have:*

$$\Pr [\langle \mathcal{P}^*, \mathcal{V}(x) \rangle = 1] \leq \epsilon(\lambda).$$

Definition 16 (Knowledge Soundness). We say that Π_{ZK} is knowledge sound if for every prover $\mathcal{P}^*$, there exists a PPT knowledge extractor $\mathcal{K}$ that has rewindable black-box access to $\mathcal{P}^*$ and a negligible function $\epsilon(\cdot)$, such that for every $(x, w) \in \mathcal{R}$, the extractor $\mathcal{K}$ on input x can output a valid witness w' for the statement x with overwhelming probability, i.e.,

$$\Pr \left[w' \leftarrow \mathcal{K}^{\langle \mathcal{P}^*(x, w), \mathcal{V}(x) \rangle}(x) : \langle \mathcal{P}^*(x, w), \mathcal{V}(x) \rangle = 1 \wedge (x, w') \notin \mathcal{R} \right] \leq \epsilon(\lambda).$$

Definition 17 (Computational Zero-Knowledge). We say that Π_{ZK} is computational zero-knowledge if for every PPT verifier $\mathcal{V}^*$, there exists a PPT simulator $\mathcal{S}$ such that for every $(x, w) \in \mathcal{R}$, the simulator $\mathcal{S}$ on input x can simulate the view of the verifier $\mathcal{V}^*$ interacting with the prover $\mathcal{P}$ with overwhelming probability, i.e., for any PPT verifier $\mathcal{V}^*$, the following two distributions are computationally indistinguishable:

$$\{\text{view}_{\mathcal{V}^*}(\langle \mathcal{P}(x, w), \mathcal{V}^*(x) \rangle)\}_{\lambda} \stackrel{c}{\approx} \{\mathcal{S}(x)\}_{\lambda}.$$

D Secure Two-Party Computation

The following description and definition follows [15].

Two-party computation. A two-party computation problem is cast by specifying a random process that maps pairs of inputs to pairs of outputs (one for each party). We refer to such a process as a *functionality* and denote it: $f : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}^* \times \{0, 1\}^*$, where $f = (f_1, f_2)$. That is, for every pair of inputs (x, y) , the output pair is a random variable $(f_1(x, y), f_2(x, y))$ ranging over pairs of strings. The first party (with input x) wishes to obtain $f_1(x, y)$ and the second party (with input y) wishes to obtain $f_2(x, y)$.

Execution in the ideal model. The security of a protocol is analyzed by comparing what an adversary can do in the protocol to what it can do in an ideal scenario that is secure by definition. An ideal execution proceeds as follows:

Inputs: Each party obtains an input, denoted w ($w = x$ for P_1 , $w = y$ for P_2).

Send inputs to trusted party: An honest party always sends w to the trusted party. A malicious party may, depending on w , either abort or send some $w' \in \{0, 1\}^{|w|}$ to the trusted party.

Trusted party answers first party: In case it has obtained an input pair (x, y) , the trusted party first replies to the first party with $f_1(x, y)$. Otherwise (i.e., in case it receives only one valid input), the trusted party replies to both parties with a special symbol $\perp$.

Trusted party answers second party: In case the first party is malicious, it may, depending on its input and the trusted party's answer, decide to stop the trusted party by sending it $\perp$. In this case, the trusted party sends $\perp$ to the second party. Otherwise, the trusted party sends $f_2(x, y)$ to the second party.

Outputs: An honest party always outputs the message it has obtained from the trusted party. A malicious party may output an arbitrary (probabilistic

polynomial-time computable) function of its initial input and the message obtained from the trusted party.

Let $f : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}^* \times \{0, 1\}^*$ be a functionality, where $f = (f_1, f_2)$, and let $\overline{M} = (M_1, M_2)$ be a pair of non-uniform probabilistic expected polynomial-time machines (representing parties in the ideal model). Such a pair is *admissible* if for at least one $i \in \{1, 2\}$ we have that M_i is honest (i.e., follows the honest party instructions in the above-described ideal execution). Then, the *joint execution of f under $\overline{M}$ in the ideal model* (on input pair (x, y)), denoted $\text{IDEAL}_{f, \overline{M}}(x, y)$, is defined as the output pair of M_1 and M_2 from the above ideal execution.

Execution in the real model. Let f be as above and let Π be a two-party protocol for computing f . Furthermore, let $\overline{M} = (M_1, M_2)$ be a pair of non-uniform probabilistic polynomial-time machines (representing parties in the real model). Such a pair is *admissible* if for at least one $i \in \{1, 2\}$ we have that M_i is honest (i.e., follows the strategy specified by Π). Then, the *joint execution of Π under $\overline{M}$ in the real model* (on input pair (x, y)), denoted $\text{REAL}_{\Pi, \overline{M}}(x, y)$, is defined as the output pair of M_1 and M_2 resulting from the protocol interaction.

Definition 18 (secure two-party computation). *Let f and Π be as above. Protocol Π is said to **securely compute f** (in the malicious model) if for every pair of admissible non-uniform probabilistic polynomial-time machines $\overline{A} = (A_1, A_2)$ for the real model, there exists a pair of admissible non-uniform probabilistic expected polynomial-time machines $\overline{B} = (B_1, B_2)$ for the ideal model, such that*

$$\{\text{IDEAL}_{f, \overline{B}}(x, y)\}_{x, y} \stackrel{c}{\approx} \{\text{REAL}_{\Pi, \overline{A}}(x, y)\}_{x, y}$$

Namely, the two distributions are computationally indistinguishable.

In this work, we require the secure two-party computation protocol to provide statistical security for one party, as defined below.

Definition 19 (secure two-party computation with statistical security for P_1). *Let f and Π be as above. Protocol Π is said to **securely compute f with statistical security for P_1** (in the malicious model) if for every pair of admissible non-uniform probabilistic polynomial-time machines $\overline{A} = (A_1, A_2)$ for the real model, there exists a pair of admissible non-uniform probabilistic expected polynomial-time machines $\overline{B} = (B_1, B_2)$ for the ideal model, such that*

$$\begin{aligned} \{\text{IDEAL}_{f, \overline{B}}(x, y)\}_{x, y} &\stackrel{s}{\approx} \{\text{REAL}_{\Pi, \overline{A}}(x, y)\}_{x, y}, \text{ if } A_1, B_1 \text{ are honest;} \\ \{\text{IDEAL}_{f, \overline{B}}(x, y)\}_{x, y} &\stackrel{c}{\approx} \{\text{REAL}_{\Pi, \overline{A}}(x, y)\}_{x, y}, \text{ otherwise.} \end{aligned}$$